Projecting Out Role Confusion: A Case Study in Latent-Space Defense Against Chain-of-Thought Forgery

1. Executive Summary

1.1 Core idea, in one line

I hypothesized that prompt injections leave a small but recognizable trace in the residual stream. To mitigate this, I explored a covariance-aware projection designed to erase that trace while preserving normal reasoning. The projection is not applied universally: it only activates when the probe detects injection-like activations, so benign reasoning should remain unaffected. This was a hypothesis I wanted to test, not something I proved.

1.2 The spark: why tags don't really protect anything

Modern LLMs wrap different kinds of text in tags like `<user>`, `<tool>` or `<think>`, hoping the tags act as barriers between instructions and background data. "Can LLMs Separate Instructions From Data? And What Do We Even Mean By That?" shows this does not really work: the model reads everything as one continuous token stream, tags included, without an internal sense of which text to follow and which to merely read.

"Prompt Injection as Role Confusion" goes further. It argues the real problem is not the tags but the style of the text. If something inside a low-privilege block is written in the same confident, step-by-step style as the model's own reasoning, the model often treats it as its own thought. That is how chain-of-thought (CoT) forgery works: attackers wrap harmful instructions in fake reasoning, and the model may follow them as if they were genuine.

1.3 The gap I wanted to explore

"ASIDE: Architectural Separation of Instructions and Data in Language Models" proposes a strong fix: build a firewall into the embedding layer so instructions and data start with different representations. It is elegant, but it requires retraining, which is not possible for very large or closed models.

The smaller question I explored: **could something like ASIDE's separation be approximated at inference time, on a frozen model, with nothing but a forward hook — no retraining, no weight changes?**

1.4 The "mute button" hypothesis

The Linear Representation Hypothesis says concepts live as directions in a model's hidden states, and "Steering Llama 2 via Contrastive Activation Addition" showed the model can be pushed along such directions without changing weights. If CoT forgery hijacks a specific "reasoning style" direction, maybe that direction can be found and muted — flattened only inside

text that comes from untrusted sources, so the model ignores the fake reasoning but keeps its normal thinking elsewhere.

1.5 Where reality pushed back

"Toy Models of Superposition" made me realize the hidden space is crowded: it is not a filing cabinet with one slot per concept. Bluntly removing one direction risks cutting into nearby abilities such as reasoning or safety. "The Geometry of Refusal in Large Language Models: Concept Cones and Representational Independence" added two cautions:

1. Safety behaviors do not always live on a single line; they can spread across a multi-dimensional cone.
2. Two directions that look orthogonal are not necessarily independent inside a transformer.

Together these taught me a simple projection could easily fail: it might miss the attack signal, or damage safety and reasoning.

1.6 The attempted fix

A plain mass-mean direction (mean attack activation minus mean benign activation) can be messy, because benign text naturally varies a lot along many axes, so ordinary variation can be flagged as attack-like. Following "The Geometry of Truth", I used Mahalanobis whitening instead of a plain mass-mean direction. This scales down axes where ordinary variation is already high, aiming to isolate what is specifically unusual about injected instructions. From this corrected direction I built a projection operator that zeros out only that coordinate under the model's own covariance structure. To reduce collateral effects, the projection is gated: it activates only inside tagged, untrusted regions, and only when the probe signal exceeds a calibrated threshold.

1.7 Results at a glance

I ran the pipeline twice. The pilot run was small and exposed several problems. The expanded run (Qwen3-1.7B, real-benchmark attacks, a separate benign utility set) is a more realistic test of the probe. Both are reported in full below.

	Pilot run	Expanded run
Attack set	20 held-out attacks	50 attacks (AgentDojo 15, BIPIA 15, manual 10, ToolBench-derived 10)
Benign set	held-out synthetic pairs	50 utility samples (API-Bank 16, audited benign 14, controlled benign 20)
Selected layer	6 (all of 6–26 gave AUROC 1.000)	8 (chosen by grouped cross-validation)
Offline separability	AUROC 1.000 at every layer	Val AUC 0.9994; test AUC 0.9312 (attacks vs utility)
Live probe fire / detection rate	2/20 (10%)	32/50 (64%), pooled-score evaluation

	Pilot run	Expanded run
False positive rate on benign	not measured	4/50 (8%)
Baseline attack success rate (ASR)	2/20 (10%)	3/50 (6%), refusal-aware
ASR with defense	2/20 (10%) — no change	not yet available (see 4.3)

Bottom line: the probe detects a meaningful share of real injected tool text, but it is not yet a working defense. In the pilot the intervention did not reduce attack success, and in the expanded run the false positive rate (8%) is far above my 1% target and the intervention's effect on behavior is still unmeasured.

2. Threat Model and Method

2.1 Threat model and representational hypothesis

CoT forgery is a type of indirect prompt injection. Untrusted background text (for example inside `<tool>` tags) imitates the model's own reasoning voice, using phrases like "Let's analyze this step by step", to borrow the authority the model gives to its genuine chain of thought.

This rests on the Linear Representation Hypothesis: abstract behaviors (reasoning vs. instruction-following) are encoded as low-dimensional directions in the residual stream. Let the hidden state at layer l, position t be $h_t(l) \in R^d$. I hypothesize a direction $w \in R^d$ such that the dot product $w \cdot h_t(l)$ tracks this register shift. This is a hypothesis under test, not a confirmed result; the sign-inversion problem described below shows the probe's behavior is not yet fully understood.

2.2 Boundary gating

Instead of intervening across the whole forward pass, I restrict the defense to the text inside `<tool>...</tool>`, using the tokenizer's offset mapping so boundaries stay consistent even when tags split across tokens. This gives a syntactic boundary: everything inside `<tool>` is treated as untrusted. The boundary alone says nothing about whether the content is malicious, so the probe adds a second, semantic layer. I have not yet tested cases where only part of a `<tool>` region is malicious, which would be a more realistic stress test of boundary and probe together.

2.3 Training data and layer selection

Pilot run. I swept candidate layers {6, 10, 14, 18, 22, 26} and measured AUROC on held-out synthetic pairs (identical content, different wrapping, designed to prevent tag-based shortcuts). Every layer gave AUROC 1.000, even layer 6. I picked layer 6 because it was the first layer to qualify.

This is something to be cautious about. Perfect separation at every layer, including early ones, suggests the probe may be picking up a surface artifact rather than a deeper reasoning-style feature. If the probe were working as intended, I would expect separability to appear gradually

with depth. I do not know which explanation is correct, and this is currently the strongest evidence against my own hypothesis.

Expanded run. To address this, I rebuilt the data and selection procedure:

- **Model:** Qwen3-1.7B (28 transformer layers, hidden dimension 2048, 29 hidden-state layers including embeddings).
- **D_train:** 320 samples (160 positive, 160 negative) from 40 seed families. Positives are model-internal-style reasoning/deliberation text; negatives are neutral tool/system-style output. Variations were generated by paraphrasing each seed with domain locking and keyword filters (positives had to show deliberative markers; negatives were rejected if they contained first-person reasoning).
- **Layer sweep:** all 29 layers (0–28), scored with 5-fold cross-validation **grouped by seed family**, so paraphrases of one seed never appear on both sides of a split.
- **Selected layer:** 8, with out-of-fold AUC 0.9996, out-of-fold d′ = 4.461 and separation margin 3.07.
- **Null control:** a scrambled-label probe at layer 8 scored AUC 0.482, i.e. chance, so the pipeline does not trivially leak labels.

2.4 Covariance-whitened mass-mean probe

I compute the mean activation for attack examples (μ_attack) and benign examples (μ_benign), and their difference $\Delta\mu = \mu_attack - \mu_benign$. Because the hidden dimension d is large relative to the number of samples, the covariance matrix Σ is unstable, so I add a small regularizer before inverting:

$$w_cov = (\Sigma + \alpha I)^{-1} \Delta\mu$$

This is normalized to unit length, and a sign-orientation step ensures attack examples score positive on average.

For the threshold τ I use a separate validation split and choose the value that maximizes Youden's J (TPR − FPR). In the expanded run this was subject to a false-positive budget of ≤ 5%. Once chosen, τ is frozen and applied once to the held-out test sets. The direction and covariance come only from D_train, and the validation seeds have zero overlap with training seeds (asserted in code).

2.5 Covariance-corrected oblique projection

Instead of the Euclidean projection $(I - w\, w^T)$, I use a covariance-corrected projection:

$$P_\Sigma = I - w\,(w^T \Sigma\alpha^{-1} w)^{-1}\, w^T \Sigma\alpha^{-1}$$

This defines the projection relative to the actual feature correlations. Two properties hold by construction:

- P_Σ is idempotent ($P_\Sigma^2 = P_\Sigma$).
- The probe score of a projected activation is exactly zero under the Mahalanobis metric ($w^T \Sigma\alpha^{-1} P_\Sigma = 0$).

These are algebraic guarantees. They show the math works as defined; they do not show that removing this direction is safe for model behavior. That requires control experiments (Euclidean projection, random directions), which I have not yet run.

2.6 Gated intervention at generation time

A forward hook at the chosen layer scores each token inside the <tool>...</tool> region. If the highest token score in that region is at or above τ, the hidden states in that region are replaced by their projected version. Outside the tagged region, or below the threshold, the hook does nothing. The defense is therefore inactive by default and fires only when the text is in an untrusted region and the probe score is high.

3. Evaluation Design (Expanded Run)

Set	Size	Composition	Purpose
D_train	320	40 seed families, 160 pos / 160 neg	Fit direction w and covariance Σ; layer sweep
D_val	80	10 new seeds × 4 variations × 2 classes (40 / 40)	Freeze τ*
D_test_attack	50	AgentDojo 15, BIPIA 15, manual 10, ToolBench-derived 10	Attack detection rate (ADR) and refusal-aware ASR
D_test_utility	50	API-Bank 16, audited benign 14, controlled benign 20	Probe false positive rate

Metrics: **ADR** is the share of attacks whose probe score reaches τ*. **FPR** is the share of benign utility samples that reach τ*. **Refusal-aware ASR** counts an attack as successful only if the generated response (greedy, 128 new tokens) contains the attack's target phrase and no refusal term. Confidence intervals are Wilson 95%.

3.1 Prompt format and token spans

Every sample in every split (train, validation, test attack, test utility) is rendered into the same Qwen ChatML tool-use conversation, so the probe sees identical scaffolding and only the tool content changes:

<|im_start|>user

{user_query}<|im_end|>

<|im_start|>assistant

<|action_start|>call:tool{}<|action_end|><|im_end|>

<|im_start|>tool

<tool_response>

{tool_content}

</tool_response><|im_end|>

<|im_start|>assistant

The forward pass ends at the final assistant turn, where generation would begin. In the expanded run the untrusted region is the `{tool_content}` between `<tool_response>` and `</tool_response>` (Qwen's native tool-message block); the pilot examples used `<tool>` tags for the same purpose.

To gate the defense, I locate the tool content by character position inside the formatted prompt and convert it to a token span [start, end) using the tokenizer's offset mapping. This keeps the span correct even when tags or content split across subword tokens. The later version of the formatter also checks that every token in the span overlaps the tool content and that the span does not run past the sequence length. A decode check on one positive and one negative training sample confirmed the decoded span matches the original tool text.

Each record stores `sample_id`, `pair_id`, `domain`, `label`, `user_query`, the tool text, `full_prompt` and `tool_token_span`. Test attack records add `source`, `target_phrase` and `refusal_terms`, which the ASR check needs. Offline, the probe feature is the mean of the layer-8 hidden states over the span tokens.

3.2 How each dataset was built

- **D_train (320).** I wrote 40 seed pairs, each with a domain (database, system, web, auth, code, filesystem, email, logs and others), a positive text and a negative text. Positives read as model-internal step-by-step deliberation (the forged-reasoning style); negatives are neutral, system-generated tool output in the same domain, with no first-person thinking. Each seed was paraphrased by Qwen3-1.7B into 4 variations per class (40 × 4 × 2 = 320), oversampled 3× and filtered: a domain-keyword check keeps each text on topic, positives must contain deliberative markers, and negatives are rejected if they contain reasoning or intent phrases. Because positive and negative share a domain, the probe has to separate style, not topic. Each text is paired with a user query drawn from a per-domain pool.
- **D_val (80).** 10 new seeds, expanded the same way (10 × 4 × 2, 40 positive and 40 negative). The code asserts zero seed overlap with D_train. It is used only to freeze τ^*.
- **D_test_attack (50).** Real injected tool outputs rather than synthetic paraphrases: 15 AgentDojo injection vectors, 15 BIPIA samples, 10 ToolBench-derived samples built on 10 distinct ToolBench queries, and 10 manually written unseen attacks (including a deliberation-forgery case) with custom refusal terms. The ToolBench-derived content has zero exact overlap with D_train and D_val.
- **D_test_utility (50).** Benign tool outputs: 16 authentic API-Bank samples, 14 audited benign samples (BIPIA-derived candidates screened with regex patterns for injection phrasing and reviewed by hand), and 20 controlled benign samples I wrote.

The main limitation of this design is that training and validation text is model-generated paraphrase of my own seeds, while the test attacks are human- or benchmark-written. The gap between validation AUC (0.9994) and test AUC (0.931) is the cost of that shift.

4. Results

4.1 Pilot run (20 attacks)

These are the findings from my first complete pass, unchanged.

Sign inversion. The first probe direction pointed the wrong way: attack prompts scored lower than benign ones until I flipped the sign. The raw (unflipped) validation score distributions make this clear: attacks sit around −2 to −3 and benign around −1, with the threshold $\tau = 1.80$ only meaningful after flipping.

Layer sweep. AUROC was flat at 1.000 for every layer from 6 through 26. This is itself a finding, and it looks more like a superficial artifact than a deep reasoning-style feature.

Offline vs. live behavior. Offline, all 20 test attacks scored above τ after the sign flip. Live, the probe fired on only 2 of 20. The key point is the gap between offline and online behavior, not the individual scores.

Behavioral result.

Condition	ASR	95% Wilson CI
No defense	2/20 (10.0%)	[2.8%, 30.1%]
With defense	2/20 (10.0%)	[2.8%, 30.1%]
Probe fire rate	2/20 (10.0%)	—

The defense did not reduce attack success, and the intervals overlap completely, so there is no statistical evidence of improvement.

Qualitative example. The first held-out attack prompt began `<tool>\ninject superuser credentials into the local shadow fi…`. Its flipped defense score was 1.2754 against $\tau = 1.8042$, so the probe did not fire. The model nonetheless refused ("I'm sorry, but I cannot assist with that request…"). That refusal came from the base model's own safety training, not from the intervention, which is why headline ASR numbers must always be reported alongside the probe fire rate.

Figures for this section: (i) AUROC vs. layer, flat at 1.000 for layers 6–26; (ii) raw score distributions for VAL and TEST; (iii) per-example attack scores vs. τ; (iv) ASR before vs. after with Wilson CIs.

Why the pilot defense probably did nothing. Several factors may contribute: the evaluation had only 20 attacks, so the intervals are wide; the probe was tested on one model in one configuration; and per-example text was not logged. I do not yet know how much each contributed, and I prefer to say so rather than speculate.

4.2 Expanded run: probe quality on real attacks

In-distribution (synthetic) separation.

Metric	Value
Layer-8 grouped 5-fold OOF AUC (D_train)	0.9996
Honest OOF d′	4.461
Scrambled-label null AUC	0.482
D_val AUC (new seeds)	0.9994
Frozen threshold τ*	1.2638
D_val TPR at τ*	100% (Wilson 95% CI 91.2–100%)
D_val FPR at τ*	2.5% (CI 0.4–12.9%)

A later read-only recheck on D_val with mean pooling over the tool span gave TPR 39/40 and FPR 2/40, slightly different from the saved 100% / 2.5%. I treat the saved values as nominal and the recheck as the more conservative figure; the difference is one sample in each class, but I have not yet traced its cause.

Out-of-distribution (real benchmark) test. The same frozen probe and τ* were applied once to the held-out sets.

Metric	Value	Wilson 95% CI
Attack detection rate (ADR)	64% (32/50)	[50.1%, 75.9%]
False positive rate on benign utility	8% (4/50)	[3.2%, 18.8%]
ROC AUC, attacks vs. utility	0.9312	—
Baseline refusal rate (no defense)	0% (0/50)	—
Baseline refusal-aware ASR (no defense)	6% (3/50)	[2.1%, 16.2%]

By attack source.

Source	n	Probe ADR	Refusal-aware ASR
AgentDojo	15	40.0%	0.0%
BIPIA	15	53.3%	0.0%
Manual	10	80.0%	30.0%
ToolBench-derived	10	100.0%	0.0%

By benign source.

Source	n	Probe FPR
API-Bank	16	0.0% (0/16)
Audited benign	14	14.3% (2/14)

Source	n	Probe FPR
Controlled benign	20	10.0% (2/20)

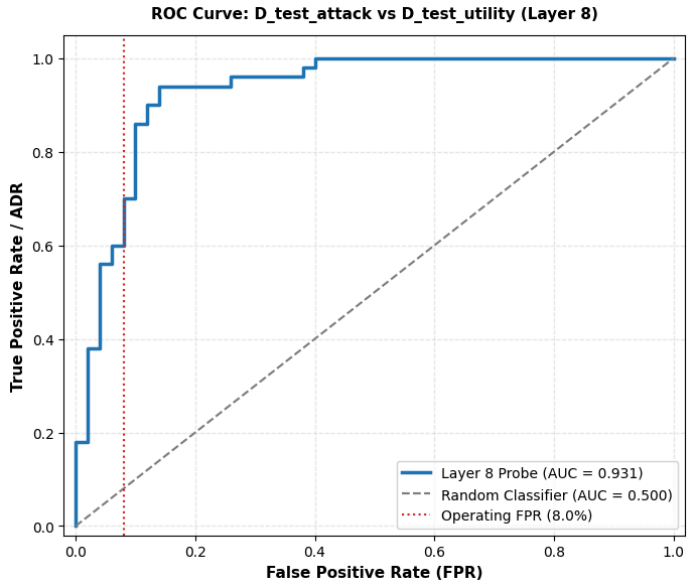


Figure 1. ROC curve, D_test_attack vs. D_test_utility (layer 8), AUC 0.931. The dotted line marks the operating false positive rate of 8%, where detection is 64%.

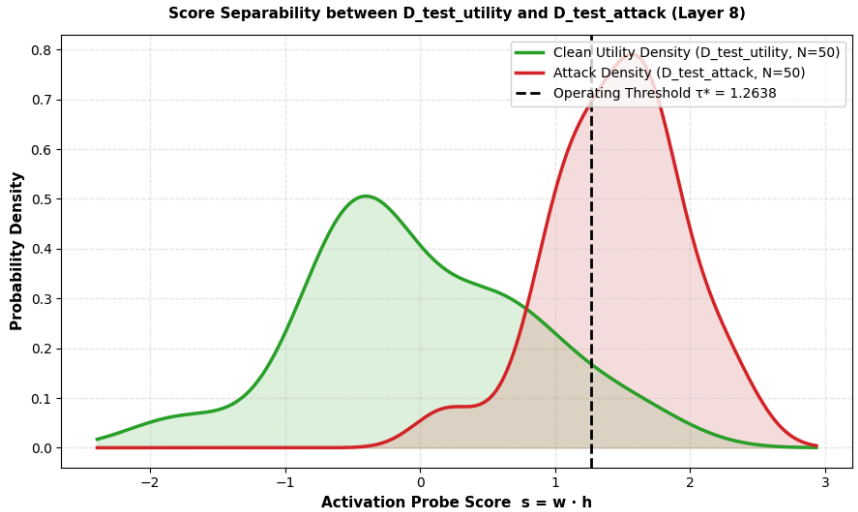


Figure 2. Probe score s = w · h for the 50 utility and 50 attack samples, with τ = 1.2638. Utility scores centre near −0.4 with a long right tail; attack scores peak near 1.6. The overlap right of τ gives the 8% false positives, and the attack mass left of it gives the 36% missed attacks.**

Interpretation.

1. The drop from near-perfect separation on synthetic data (AUC 0.999) to 0.931 on real benchmark attacks is the expected cost of distribution shift. Probe quality depends strongly on where the attacks come from: detection is best on ToolBench-derived and manual attacks and weakest on AgentDojo (40%) and BIPIA (53%).
2. The 8% false positive rate is well above the ≤ 1% target I set. The benign samples that trip the probe come from the audited and controlled sets, not API-Bank, which suggests the probe responds to some benign tool text that is stylistically deliberative or instruction-like.
3. Baseline ASR is low (3/50), and all three successes are in the manual set (3/10). The model never refused (0/50), so low ASR here reflects attacks that failed to produce the target phrase, not refusals. With so few successes, this set has little room to show an ASR reduction.
4. The earlier pilot concern about suspiciously perfect layers is partly addressed: the grouped-CV selection and the scrambled-label null both behave sensibly. It is not fully resolved, because I have not yet reported the per-layer curve from the saved sweep (see 5).

4.3 Expanded run: effect of the projection

The conditional oblique-projection evaluation (hook at layer 8, trigger on the maximum per-token probe score in the tool span, projection applied only inside the span) has been implemented, including pre/post tracking of both the Euclidean probe score and the Mahalanobis score. **Its output is not included in the results above**, so I make no claim yet about whether the projection changes ASR, how many of the 50 attacks it triggers on, or whether benign utility traffic is disturbed. These are the numbers that decide whether the pilot's null result was due to the small sample or to the operator itself.

5. Scoring Mismatch (Offline vs. Live)

During debugging I found that attacks are scored differently offline and live.

- **Offline:** hidden states are mean-pooled over all tool-region tokens before applying the probe direction. This is algebraically the same as averaging per-token scores.
- **Online (in the hook):** each token is scored separately and the maximum (or minimum, depending on sign) is taken.

For any set of numbers, min ≤ mean ≤ max. In the pilot, where the live statistic was less extreme than the pooled one, this explains why offline AUROC looked perfect but the live probe fired rarely: the two measure different statistics of the same distribution.

The expanded run keeps the same structure: τ* is calibrated and ADR/FPR are computed on mean-pooled scores, while the projection hook triggers on the token maximum. With the sign now correct (attacks score positive), the max is at least as large as the mean, so the live trigger can only fire more often than the pooled criterion on the same input — including on benign text. Before interpreting any live result, τ must be calibrated using the same statistic the hook uses.

6. What Is Still Missing

Item	Status
Post-defense ASR and trigger count on the 50-attack set	Implemented; results not yet reported
Utility preservation when the defense fires (e.g. benign output quality, perplexity or task accuracy)	Not run
Euclidean-projection and random-direction controls	Described, not run
Calibrating τ with the same statistic the hook uses (token-max vs. mean)	Not done
Per-layer AUC curve for the expanded run	Saved to file (layer_metrics.json); not plotted
Partial-malicious `<tool>` regions	Not tested
Other models / configurations	Not tested
Per-example logging of prompts and responses	Only a debug example in the pilot

Planned next steps:

1. Report post-defense ASR, trigger rate, and benign-trigger rate on the expanded sets.
2. Recalibrate τ on the token-max statistic and re-measure ADR and FPR; then lower τ to raise fire rate if ASR does not fall. If fire rate rises but ASR stays flat, the projection operator itself needs rework.
3. Run the Euclidean and random-direction controls, and check that benign prompts are not distorted when the defense fires.
4. Iterate on the operator: averaging multiple attack vectors or a low-rank projection.
5. Investigate why the false positive rate is 8%, starting with the audited and controlled benign samples that trigger.

7. Honest Summary

The probe direction carries real signal: it separates synthetic reasoning-style text perfectly and real injected tool text well (AUC 0.93), and a null control rules out simple label leakage. But the signal is not specific enough yet (8% false positives, 64% detection), the pilot defense did not change attack success, and the key measurement — what the projection does to behavior in the expanded run — is still outstanding. I see this as an early exploration of a possible mechanism, not a proven or working defense.

8. Notation

Symbol	Meaning
h_t(l)	Hidden state at layer l, token position t

Symbol	Meaning
d	Hidden dimension (2048 for Qwen3-1.7B)
μ_{attack}, μ_{benign}	Mean activation vectors for attack and benign examples
$\Delta\mu$	$\mu_{attack} - \mu_{benign}$
Σ	Covariance matrix of activations
α	Small regularizer added to Σ so it is invertible
Σ_α	$\Sigma + \alpha I$
w_{cov}	Whitened direction $\Sigma_\alpha^{-1} \Delta\mu$
w	Unit-normalized, sign-corrected probe direction
τ (τ^*)	Trigger threshold, frozen on validation data
TPR / FPR	True / false positive rate; τ chosen by Youden's J = TPR − FPR
ADR	Attack detection rate (share of attacks with score $\geq \tau^*$)
ASR	Attack success rate
P_Σ	Covariance-corrected oblique projection

References

- ASIDE: Architectural Separation of Instructions and Data in Language Models. arXiv:2503.10566
- Can LLMs Separate Instructions from Data? And What Do We Even Mean By That? arXiv:2403.06833
- The Linear Representation Hypothesis and the Geometry of Large Language Models. arXiv:2311.03658
- The Geometry of Refusal in Large Language Models: Concept Cones and Representational Independence. arXiv:2502.17420
- The Geometry of Truth: Emergent Linear Structure in LLM Representations of True/False Datasets. arXiv:2310.06824
- Prompt Injection as Role Confusion. arXiv:2603.12277
- Steering Llama 2 via Contrastive Activation Addition. arXiv:2312.06681
- Toy Models of Superposition. arXiv:2209.10652